ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

Query with open knowledge bases

Yiru Zhang

Mathematics for machine learning

Département d'Informatique

Ecole Supérieure d'Ingénieurs Léonard de Vinci

DE VINCI
RESEARCH
CENTER

Open Knowledge bases

ESILV

What Is an Open Knowledge Base?

- Key properties:
  - Open access / open license
  - Persistent URIs
  - Interlinkable (Linked Open Data)
  - Often SPARQL-queryable
- Utilisations
  - Reuse instead of rebuilding
  - Entity alignment
  - Knowledge enrichment
  - Reasoning and inference
  - Grounding for RAG / LLMs

An open knowledge base is a structured, publicly accessible repository of entities and relations, typically represented as a knowledge graph.

ESILV

Major Open Knowledge Bases

Different KBs serve different purposes.

Knowledge Base	Domain	Access
DBpedia	General	RDF / SPARQL
Wikidata	General	RDF / SPARQL
YAGO	General	RDF / SPARQL
GeoNames	Geography	API + RDF
OpenStreetMap	Spatial	API
MusicBrainz	Music	API + RDF
UniProt / GO	Biology	RDF / SPARQL

ESILV

The DBpedia ontology

- DBpedia is **automatically extracted** from Wikipedia.
  - It is **RDF**-native, with clear **ontology**
  - Human-readable URIs
- Architecture:
  - Entities (dbr:) (instances)
  - Ontology (dbo:) (classes and relations)
  - Extracted properties (dbp:)
  - Links to other KBs

dbr:Paris
dbo:City
dbo:populationTotal

Wikidata (compared with DBpedea)

- Wikidata **Manually curated** by humans and bots
  - Facts are added, corrected, and discussed
- While DBpedia prioritizes coverage (automatically extracted), Wikidata prioritizes correctness.
- Dbpedia uses human-readable URIs, while Wikidata use numeric, language-independent IDs. So DBpedia URIs are intuitive, Wikidata IDs are stable and unambiguous
  - Example

dbr:Paris
dbr:Marie_Curie

wd:Q90 (Paris)
wd:Q7186 (Marie Curie)

```
#Use owl:sameAs to bind the entity
SELECT ?wikidata WHERE {
  dbr:Paris owl:sameAs ?wikidata .
  FILTER CONTAINS(STR(?wikidata), "wikidata.org")
}
```

Wikidata ontology, schema and data quality (compared with DBpedea)

- Ontology and schema
  - **DBpedia** Uses a **lightweight ontology** (dbo:), its Classes and properties are explicitly defined
  - Wikidata is schema-light, property-centric. Its properties are entities themselves

dbo:City
dbo:birthPlace

P31 (instance of)
P19 (place of birth)

- Data Quality
  - **Dbpedia:** noisy due to automatic extraction; Incomplete or incorrect facts are possible; Ontology mismatches occur
  - **Wikidata:** data quality is higher, validated by the community; There are explicit references to sources
  - If correctness matters, prefer Wikidata.

ESILV

Query over Dbpedia and wikidata

- Both provide public SPARQL end points
- They are both build on RDF data model
- Some differences

Aspect	DBpedia	Wikidata
Ontology clarity	Higher	Lower
Query complexity	Simpler	More complex
Documentation	Ontology-driven	Property-driven

- We will see the query later.

ESILV

Query with DBpedea

SPARQL Query with DBpedia

- Demo: https://dbpedia.org/sparql
- DBpedia is linked with Wididata by owl:SameAs

```
#With LIMIT
SELECT ?city WHERE {
  ?city rdf:type dbo:City .
}
LIMIT 10
```

```
# Adding Constraints
SELECT ?city WHERE {
  ?city rdf:type dbo:City ;
        dbo:country dbr:France .
}
```

```
# With Literals
SELECT ?city ?pop WHERE {
  ?city rdf:type dbo:City ;
        dbo:country dbr:France ;
        dbo:populationTotal ?pop .
  FILTER (?pop > 1000000)
}
```

```
# OPTIONAL pattern
SELECT ?city ?area WHERE {
  ?city rdf:type dbo:City .
  OPTIONAL { ?city dbo:areaTotal ?area }
}
LIMIT 10
```

```
# Aggregation
SELECT (COUNT(?city) AS ?numCities) WHERE {
  ?city rdf:type dbo:City ;
        dbo:country dbr:France .
}
```


ESILV

Ambiguity in Entity Lookup

- Ambiguity Example:
  - Paris (France)
  - Paris (Texas)
  - Paris (mythology)
  - Paris (person)
- Entity alignment requires context.
- Some strategies:
  - Graph-Level Context-based filtering
  - Type-based disambiguation
  - Property Evidence: Match known attributes, such as population, coordinates, birth date, country. More matching properties -> higher confidence

```
SELECT ?entity WHERE {
  ?entity rdfs:label "Paris"@en ;
        dbo:country dbr:France ;
        rdf:type dbo:City .
}
```

```
?entity rdf:type dbo:City .
```


ESILV

Handle ambiguity Using DBpedia Spotlight

- **DBpedia Spotlight** is an open-source tool that identifies and links named entities in text to their corresponding entries in DBpedia.
- Use API request
  - Send your text to Dbpedia Spotlight API with HTTP GET (e.g., " Paris is the capital of France. Paris, Texas is a 1984 neo-Western drama road film directed by Wim Wenders. Paris Whitney Hilton is an American media personality, businesswoman, and socialite. ")
  - Make sure to set the appropriate HTTP headers (e.g., Accept: application/json).

```
curl 'https://api.dbpedia-spotlight.org/en/annotate?text=%E2%80%9CParis%20is%20the%20capital%20of%20France.%22confidence=0.5' -H "Accept: application/json"
```


ESILV

Demo of spotlight

Once we have the Dbpedia URI, we can map it into a unique identifier (e.g. with wikidata)


```
{
    "@text": "“Paris is the capital of France.\"",
    "@confidence": "0.5",
    "@support": "0",
    "@types": "",
    "@sparql": "",
    "@policy": "whitelist",
    "Resources":
    [
        {
            "@URI": "http://dbpedia.org/resource/Paris",
            "@support": "151504",
            "@types": "
                Wikidata:Q486972,Schema:Place,DBpedia:PopulatedPlace,DBpedia:Place,DBpedia:Location,DBpedia:Settlement",
            "@surfaceForm": "Paris",
            "@offset": "1",
            "@similarityScore": "0.9999270804856535",
            "@percentageOfSecondRank": "7.238654819670028E-5"
        },
        {
            "@URI": "http://dbpedia.org/resource/France",
            "@support": "240219",
            "@types": "Wikidata:Q6256,Schema:Place,Schema:Country,DBpedia:PopulatedPlace,DBpedia:Place,DBpedia:Location,DBp
            edia:Country",
            "@surfaceForm": "France",
            "@offset": "25",
            "@similarityScore": "0.9971086281130809",
            "@percentageOfSecondRank": "9.188299638264375E-4"
        }
```

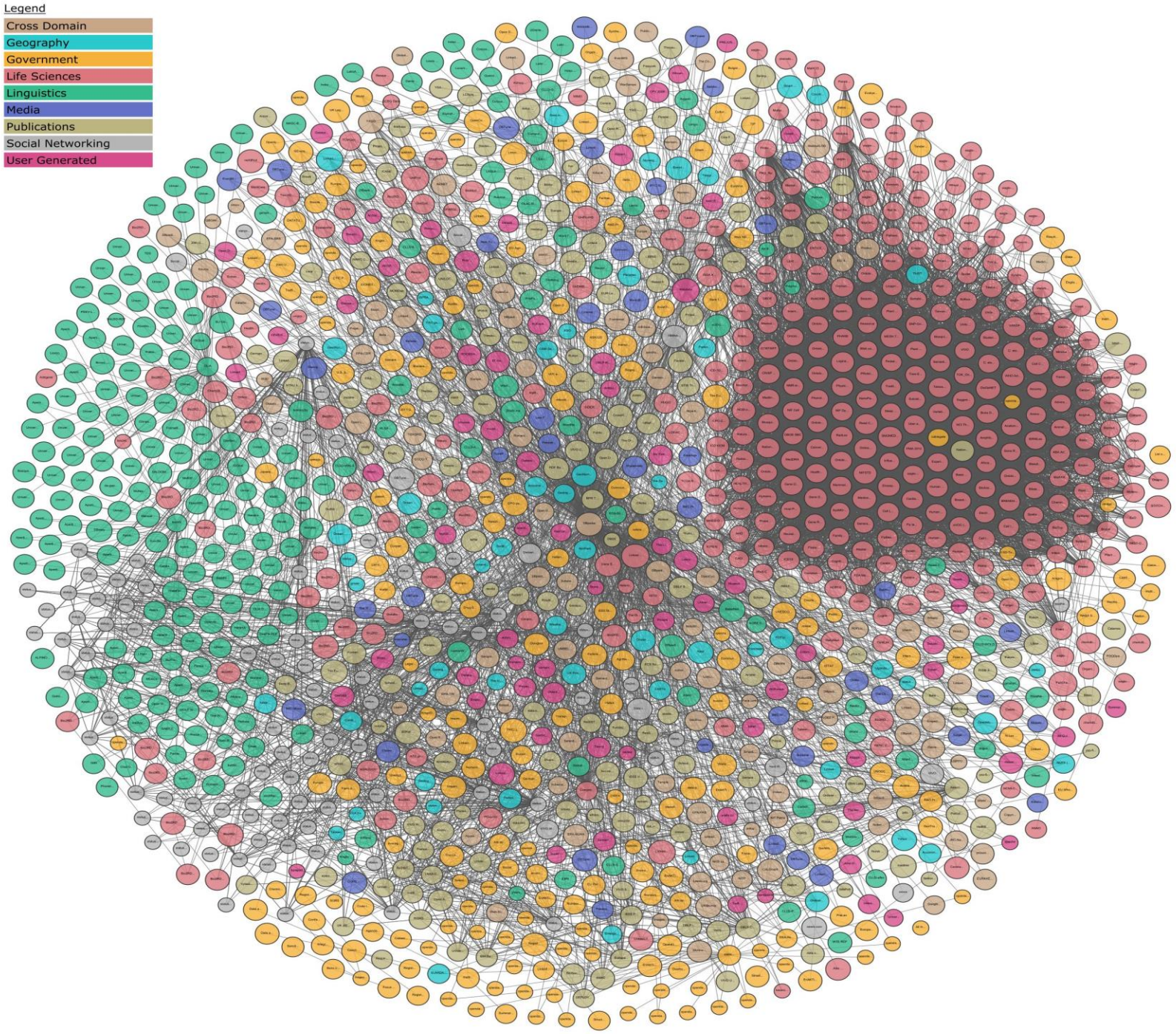
Programming with Wikidata?

- It is also possible to do entity aligning/linking to wikidata Q-IDs, with qwikidata, or directly through http request to wikidata api https://www.wikidata.org/w/api.php

- No much difference from DBpedia. I leave you learn by your own through your project.

ESILV

The linked open data cloud



The Linked Open Data Cloud from lod-cloud.net

The Linked Open Data

- **Linked Open Data (LOD)** is a method of publishing structured data so that it can be interlinked and become more useful through semantic queries.
- Core characteristics: Open access, Structured (RDF), Identifiable (URIs), Interlinked
- It uses HTTP URIs to name things, and provide useful information (SPARQL query).
  - RDF is the datamodel behind LOD.
  - URIs are global identifiers.
- Linking is explicit.
  - owl:sameAs (identity)
  - skos:exactMatch
  - rdfs:seeAlso

dbr:Paris dbo:country dbr:France .

http://dbpedia.org/resource/Paris

dbr:Paris owl:sameAs wd:Q90 .

ESILV

The Linked Open Data Cloud

- https://lod-cloud.net/
- LODC consist of Hundreds of datasets, with Cross-domain coverage, over Billions of triples
- Dbpedia and Wikidata are the core

Dataset	Domain	Notes
DBpedia	General	Wikipedia-derived
Wikidata	General	Human-curated
GeoNames	Geography	Spatial entities
MusicBrainz	Music	Artists, albums
UniProt	Biology	Protein data

ESILV

How Knowledge Graphs Interact

- Interaction mechanisms:
  - Shared URIs
  - Semantic links
  - Schema alignment
  - Federated queries
- The Key Linking Predicate: owl:sameAs

dbr:Paris owl:sameAs wd:Q90 .

Knowledge Graph	URI
DBpedia	`dbr:Paris`
Wikidata	`wd:Q90`
GeoNames	`gn:2988507`

- Alignment enables cross-graph reasoning and querying.

ESILV

Federated SPARQL Queries

- SPARQL supports querying multiple knowledge base (endpoint), with SERVICE:
- Demo
  - https://query.wikidata.org/
  - RDFLib
- Result:
  - Population from DBpedia
  - Coordinates from Wikidata

```
PREFIX wdt:  <http://www.wikidata.org/prop/direct/>
PREFIX wd:   <http://www.wikidata.org/entity/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
PREFIX dbr:  <http://dbpedia.org/resource/>
SELECT ?pop ?coords
WHERE {
   SERVICE <https://dbpedia.org/sparql> {
     dbr:Paris dbo:populationTotal ?pop .
   }
   SERVICE <https://query.wikidata.org/sparql> {
     wd:Q90 wdt:P625 ?coords .
   }
}
```

Table | 1 résultat en 1002ms | Code | Télécharger | Lien

pop	coords
2048472	Point(2.352222222 48.856666666)

How to Specify / Restrict a Namespace in SPARQL?

```
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbr: <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

- Sometimes you want: “Only relations defined in DBpedia ontology”
  - You should filter with the entire namespace, rather than only the prefix

```
SELECT ?s ?p ?o WHERE {
  ?s ?p ?o .
  FILTER(STRSTARTS(STR(?p), "http://dbpedia.org/ontology/"))
}
```


ESILV

Linked Open Data Matters for AI

- LOD provides structured background knowledge with Entity grounding.
- These facts make the model explainable and interoperability
- A bridge between LOD and modern AI (LLM specifically)
  - LOD → Knowledge Graphs → **Embeddings** → **RAG**
  - (We'll learn embeddings and RAG later)

ESILV

LOD vs Classical Databases

Aspect	LOD	Relational DB
Schema	Flexible	Fixed (defined when creating the tables)
IDs	Global URIs	Local keys
Linking	Native	External (join with keys)
Evolution	Easy	Costly

ESILV

How it helps in the project?

ESILV

Recall: Process of knowledge base/graph building

1. Representing Extracted Information as Triples (RDF triples) ✅
2. Defining an Ontology(Using RDF, RDFS, OWL, etc.) ✅
3. Data Integration and Normalization
   - Entity linking with **external knowledge bases**
4. Storing the Knowledge Graph
   - Graph databases and RDF stores
5. Populating the Knowledge Graph ✅
   - Write scripts or use ETL tools to convert NER and RE into triples
6. Querying and Enriching the Graph
   - SPARQL Query ✅

ESILV

Data Integration and Normalization

- Data integration and normalization is a critical step in building a high-quality knowledge graph.
- Identical entity often have variations
  - e.g., "IBM," "International Business Machines," "IBM Corp."
- Objective: resolve these variations and map them to unique, canonical representations.

ESILV

Entity Resolution / Linking

- Purpose: Merge different mentions of the same entity into a single canonical entity.
- Techniques:
  - **String Similarity & Matching:**
    - Use similarity measures (like Levenshtein distance or **cosine similarity** on **word embeddings**) to identify similar entity mentions.
  - **Contextual Clues:**
    - Analyze surrounding text or use features like entity type, part-of-speech tags, and syntactic context to disambiguate entities.
  - **Knowledge Base Mapping:**
    - Use external knowledge bases (e.g., DBpedia, Wikidata, or Freebase) to link extracted entities to canonical identifiers.
    - Tools like **DBpedia Spotlight** or **TAGME** can automate this mapping.

ESILV

Integration into the Knowledge Graph/Base

- Once data is normalised, we can build the knowledge graph.
- Use RDF libraries, such as Apache Jena (in Java) and RDFLib (in Python). The knowledge graphs are stored via rdf files.
- The knowledge graph can be stored using Graph databases, like Neo4j, GraphDB, etc. They allow to store and manage the knowledge graph with database management system feature, such as optimised query, back-up, conflict management, distributed storage, etc. (You will have graph mining module with Prof. Nicolas Travers)

ESILV

Integrate private knowledge base with open knowledge bases

- Integrate your private KB with open KBs are very helpful, by making your own knowledge **contextualized**, **interoperable**, and **reusable.**
- By linking to open KBs. Your entities gain rich semantic descriptions and external attributes become accessible.
  - Eg, in your private KB, there is an entity ex:ProjectX. It is indeed the city Paris. However, there is no much semantic information about it, just the capital of France.
  - By adding ”ex:ProjectX owl:sameAs dbr:Paris .” as a fact, your private data is no longer semantically isolated!
- Always **Link** to Open facts rather than **duplication!**

ESILV

Linking, not duplicating!

- Bad example:

```
ex:ParisOffice
  dbo:populationTotal 2148000 ;
  dbo:country dbr:France .
```

  - The population and country are COPIED from DBpedia to your private KB
- Good Practice:

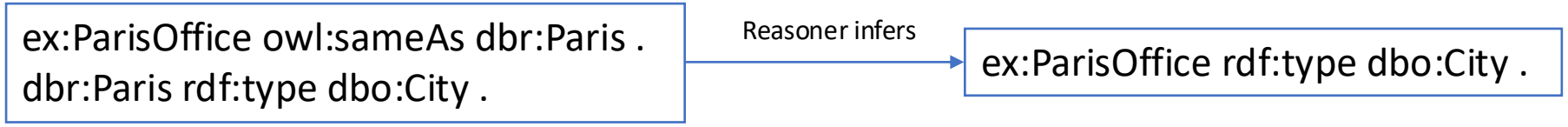
```
ex:ParisOffice owl:sameAs dbr:Paris .
```

  - DBpedia remains the authority and your KB
- Why doing so?
  - Avoid data inconsistency (e.g. when Dbpedia updates population, your KB **outdates** and **conflicts** with **duplicate**)
  - Preserve authority and provenance (your implicitly claim authority if you duplicate)
  - Next slide.

ESILV

Advantages of linking (not dupicating).

- (Following last slide)
  - Reduce Maintenance Cost and avoid semantic drift (updates are synchronized automatically)
  - Enable reasoning and inference



  - Control privacy and exposure (linking exposes only the identity, while duplicating risks leaking public structure into private data and blurs public/private boundary)
  - Query Power increases (linking allows federate query across dataset, and multi-hop questions are possible. E.g., “Which internal offices are in capital cities of countries with GDP > X?”)
  - Better AI and RAG behaviours

ESILV

What to do with the predicates (relations)

- For the relations, also avoid duplicating (avoid creating your own relation if not necessary)
- Use existing relations
  - Avoid: dbr:Paris ex:hasPopulation 2148000 .
  - Use: dbr:Paris dbo:populationTotal 2148000 .
- Create a new relation only when the relation does not exist, or has different semantics (e.g. you have an estimated population defined as ex:estimatedPopulation, which is not the official population usually defined by dbo:populationTotal)
  - When creating a new relation, **link it semantically**

ex:estimatedPopulation rdfs:subPropertyOf dbo:populationTotal .

ESILV

How to integrate?

- Assume that you have already build your private knowledge, with ontology and private URI namespace. You can simply add them as additional triplet to an open KB.
- But sometimes, the open KB is too large (DBpedia, Wikidata and YAGO include the entire Wikipedia…)
- Just find the neighbourhood subgraph from the open KB
  - **Entity Linking:** Convert your text strings into standard IDs (in the open KB)
  - **Subgraph Extraction:** Query the Open KB to get the "neighbors" of your entities. This turns your 100 triplets into perhaps 2,000–5,000 triplets.
  - **Merge**: Combine your proprietary triplets with the fetched public triplets.
- (You need to do this in your project)

ESILV

Examples: start with 100 triplets

- Entity linking through APIs, get URI in DBpedia (dbr:xxx) or in wikidata (wd:xxx).
- Anchoring: extract neighbourhoods
  - Fetch the surrounding knowledge from DBpedia or wikidata, based on your 100 triplets, to increase the graph entity, with SPARQL queries
  - You can do this in an iterative way so that multi-hop neighbours are fetched.
- Merge your triplets into the neighbourhood
  - Use existing relations if exists
  - Check if the fact triplets exist already in the graph
    - Add new facts only one fact does not exist

```
# example:
SELECT ?p ?o WHERE {
{
wd:{entity_id} ?p ?o .
}
}
LIMIT 50
```

TRY WITH YOUR DATA!